



Manual Técnico

APLICACIÓN WEB DE RÉCORD DE CATEDRÁTICOS Y GESTIÓN DE CURSOS

DATOS DE LOS MIEMBROS DEL GRUPO #5

NOMBRE	CARNÉ
Jason Eliezer Miculax Cacatzí	201903829
Diego Alejandro Luna de la Cruz	202104231
Javier Alejandro Marroquín Canizales	202402017
Perla Alejandra García Ordóñez	202300343
Ingrid Carolina Pineda Pérez	202300661
Jasson Enrique Cabrera Estrada	202100047
Ana Lucía Aguilar Mendía	202500679

TUTORES A CARGO

Dorbal Emilio Aldana Ramos
Alex Ricardo Castañeda Rodríguez
Alex Oswaldo López Alquejay

ÍNDICE

INTRODUCCIÓN	3
OBJETIVOS	4
General	4
Específicos.....	4
1. ESPECIFICACIONES TÉCNICAS	5
2. LÓGICA DE LA APLICACIÓN	6
Paquetes Utilizados.....	6
BACKEND.....	7
/src	7
/.env	22
/package.json.....	22
/package-lock.json	22
/node_modules	22
FRONTEND	22
/Public.....	22
/Src.....	23
/index.html	30
/.gitignore	30
/.oxlintrc.json	30
/package.json.....	30
/package-lock.json	31
/README.md	31
/vite.config.js	31
/Node_modules	31
GUÍA DE EJECUCIÓN Y DESPLIEGUE DEL SISTEMA.....	31
3. SECCIÓN DE AYUDA Y RESOLUCIÓN DE PROBLEMAS (TROUBLESHOOTING).....	33
CONCLUSIONES.....	34
REFERENCIAS	35

INTRODUCCIÓN

El presente Manual Técnico documenta de manera detallada la arquitectura de software, la configuración del servidor backend y la estructura de la base de datos para la aplicación web de evaluación docente y control académico de la Escuela de Ciencias y Sistemas. La capa del servidor se encuentra desarrollada sobre el entorno de ejecución Node.js utilizando el framework Express, bajo una arquitectura Cliente-Servidor orientada a servicios API REST. Esta infraestructura desacoplada permite procesar las peticiones HTTP del cliente de forma asíncrona y gestionar la persistencia de datos mediante un sistema gestor relacional MySQL, garantizando el flujo continuo de información entre el frontend y el servidor.

Por otro lado, este documento especifica el diseño del modelo entidad-relación, la definición formal de los endpoints de la API, las políticas de seguridad (como encriptado de credenciales y configuración de CORS) y el despliegue del entorno de desarrollo. Este manual está dirigido tanto al equipo de desarrollo como a los evaluadores académicos del curso, funcionando como un instrumento de auditoría y guía de mantenimiento que facilita la verificación del código fuente, la resolución de problemas técnicos y la futura escalabilidad del sistema.

OBJETIVOS

General

- Documentar la arquitectura técnica, el modelo relacional de datos y la especificación de los endpoints de la API REST para garantizar la mantenibilidad, escalabilidad y correcta evaluación del servidor web.

Específicos

- Detallar la estructura de las tablas y relaciones en MySQL para el almacenamiento de usuarios, publicaciones, comentarios y cursos aprobados.
- Especificar los contratos de servicio de la API REST (métodos HTTP, rutas y parámetros) construida sobre Node.js y Express.
- Proporcionar una guía de solución de problemas (Troubleshooting) y referencias técnicas para la puesta en marcha del entorno servidor.

1. ESPECIFICACIONES TÉCNICAS

HADWARE

- Procesador: Intel Core i3 de 2.0 GHz o equivalente (se recomienda Core i5 o superior).
- Memoria RAM: 4 GB mínimo (8 GB recomendados para un mejor rendimiento en VS Code y ejecución simultánea de backend y frontend).
- Espacio en Disco: 200 MB para instalación de Node.js y VS Code.
- 500 MB adicionales para dependencias y librerías de Node.js.
- 200 MB para dependencias del frontend (React + Vite).
- Disco duro: 2 GB libres como mínimo.
- Pantalla: Resolución mínima de 1280x720 (se recomienda 1366x768 o superior).
- Periféricos: Teclado y mouse (recomendado).

SOFTWARE

- Sistema Operativo:
 1. Windows 10/11 (64 bits recomendado).
 2. GNU/Linux (Ubuntu 20.04 en adelante).
 3. macOS.
- Lenguaje de Programación:
 1. JavaScript (ECMAScript 6 o superior).
 2. JSX (para el desarrollo con React).
- Entorno de ejecución:
 1. Node.js versión 18 o superior (incluye npm versión 9 o superior).
- Editor de Código:
 1. Visual Studio Code versión 1.70 o superior.
- Frameworks y librerías utilizadas
 - Backend:
 1. Express (para creación de la API REST).
 2. Nodemon (para reinicio automático del servidor en desarrollo).
 3. MySQL2 (para conexión con la base de datos).
 - Frontend:
 1. React (para el desarrollo de la interfaz de usuario).
 2. Vite (como entorno de construcción y servidor de desarrollo).
 3. @vitejs/plugin-react (para integración de React con Vite).
 - Base de Datos:
 1. MySQL versión 8.0 (local o en la nube de docker hub).

2. LÓGICA DE LA APLICACIÓN

Paquetes Utilizados

Nombre del Paquete	Descripción	Objetos dentro del Paquete
Backend	Paquete que contiene la arquitectura del servidor Node.js y Express. Gestiona la lógica de negocio, la comunicación con MySQL, las consultas de usuarios, cursos, catedráticos, publicaciones, comentarios y asignaciones académicas.	Carpeta src: • app.js • Carpeta src/config/: • database.js • Carpeta src/controllers/: • usuarioController.js • catedraticoController.js • cursoController.js • publicacionController.js • comentarioController.js • cursoAprobadoController.js • Carpeta src/routes/: • usuarioRoutes.js • catedraticoRoutes.js • cursoRoutes.js • publicacionRoutes.js • comentarioRoutes.js • cursoAprobadoRoutes.js package-lock.json Package.json
Fronted	Paquete que gestiona la interfaz de usuario (UI) desarrollada en React.js. Contiene los componentes, páginas y estilos necesarios para interactuar con los endpoints expuestos por el backend.	Carpeta public/: • favicon • icons Carpeta src/: • Carpeta components/: • Navbar.jsx • PublicacionCard.jsx • Carpeta context/: • AuthContext.jsx • Carpeta pages/: • BuscarUsuario.jsx • CrearPublicacion.jsx • CursosAprobados.jsx • Inicio.jsx • Login.jsx • Perfil.jsx • RecuperarPassword.jsx • Registro.jsx • Carpeta services/: • api.js • Carpeta styles/ • App.css • App.jsx • index.css

		• main.jsx Archivos Raíz: index.html .gitignore oxlintrc.json package.json package-lock.json README.md vite.config.js node_modules
--	--	--

BACKEND

CLASES Y MÓDULOS UTILIZADOS

/src

src/app.js

Este archivo constituye el punto de entrada principal del servidor Express. En él se configuran los middlewares globales del sistema, la conexión inicial a la base de datos MySQL, el enrutamiento principal de la API REST y el manejo centralizado de excepciones y rutas no encontradas:

- Middlewares Integrados: Implementa **cors()** para permitir peticiones de origen cruzado desde el frontend y **express.json()** para el procesamiento automático de cuerpos de solicitud en formato JSON.
- Diagnóstico de Conexión: Expone el **endpoint /api/prueba-db** que ejecuta una consulta de prueba (**SELECT 1**) sobre el pool de MySQL para verificar la disponibilidad del servicio.
- Enrutamiento Centralizado: Monta los módulos de rutas bajo el prefijo **/api** para los recursos del sistema (**/usuarios, /cursos, /catedraticos, /publicaciones, /comentarios y /cursos-aprobados**).
- Manejo Global de Errores: Incorpora un interceptor para respuestas 404 (Ruta no encontrada) y un middleware de error con firma (**error, req, res, next**) que captura fallos no controlados devolviendo un estado HTTP 500.

```

1  const express = require('express');
2  const cors = require('cors');
3
4  const pool = require('./config/database');
5
6  const usuarioRoutes = require('./routes/usuarioRoutes');
7  const cursoRoutes = require('./routes/cursoRoutes');
8  const catedraticoRoutes = require('./routes/catedraticoRoutes');
9  const publicacionRoutes = require('./routes/publicacionRoutes');
10 const comentarioRoutes = require('./routes/comentarioRoutes');
11 const cursoAprobadoRoutes = require('./routes/cursoAprobadoRoutes');
12
13
14 const app = express();
15
16
17 // =====
18 // MIDDLEWARES
19 // =====
20
21 app.use(cors());
22
23 app.use(express.json());
24
25
26 // =====
27 // RUTA PRINCIPAL
28 // =====
29
30 app.get('/', (req, res) => {

```

```

31
32     res.json({
33         mensaje: 'API de Desarrollo Web funcionando correctamente'
34     });
35
36 });
37
38
39 // =====
40 // PRUEBA DE MYSQL
41 // =====
42
43 app.get('/api/prueba-db', async (req, res) => {
44
45     try {
46
47         const [resultado] = await pool.query(
48             'SELECT 1 AS conectado'
49         );
50
51
52         res.json({
53
54             mensaje: 'Conexión con MySQL funcionando correctamente',
55

```

```

124 // =====
125 // MANEJO DE RUTAS NO ENCONTRADAS
126 // =====
127
128 app.use((req, res) => {
129
130     res.status(404).json({
131
132         mensaje: 'Ruta no encontrada',
133
134         ruta: req.originalUrl
135
136     });
137
138 });
139
140
141 // =====
142 // MANEJO GENERAL DE ERRORES
143 // =====
144
145 app.use((error, req, res, next) => {
146
147     console.error('Error del servidor:', error);
148
149
150     res.status(500).json({
151
152         mensaje: 'Error interno del servidor',

```

```

153
154         error: error.message
155
156     });
157
158 });
159
160
161 // =====
162 // SERVIDOR
163 // =====
164
165 const PORT = process.env.PORT || 3000;
166
167
168 app.listen(PORT, () => {
169
170     console.log(
171         `Servidor ejecutándose en http://localhost:${PORT}`
172     );
173
174 });

```


src/config

database.js: este módulo es responsable de establecer y administrar la conexión técnica entre la aplicación Node.js y el motor de base de datos MySQL mediante la librería `mysql2/promise`:

- **Uso de Promesas:** Emplea el cliente basado en promesas (**mysql2/promise**), lo que permite realizar consultas asíncronas de manera eficiente utilizando la sintaxis **async/await** en los controladores.
- **Variables de Entorno:** Carga credenciales sensibles (**DB_HOST, DB_PORT, DB_USER, DB_PASSWORD y DB_NAME**) de forma segura a través de `dotenv`, evitando exponer datos críticos directamente en el código fuente.
- **Pool de Conexiones:** Configura un grupo de conexiones reutilizables (**createPool**) con un límite de hasta 10 conexiones simultáneas (**connectionLimit: 10**) y espera activa (**waitForConnections: true**), optimizando los recursos del servidor al no tener que abrir y cerrar una conexión por cada solicitud HTTP.

```
1  const mysql = require('mysql2/promise');
2  require('dotenv').config();
3
4  const pool = mysql.createPool({
5    host: process.env.DB_HOST,
6    port: process.env.DB_PORT,
7    user: process.env.DB_USER,
8    password: process.env.DB_PASSWORD,
9    database: process.env.DB_NAME,
10
11    waitForConnections: true,
12    connectionLimit: 10,
13    queueLimit: 0
14  });
15
16  module.exports = pool;
```

Configuración del pool de conexiones a MySQL

src/ routes.js

La capa de enrutamiento actúa como el intermediario que mapea las solicitudes HTTP entrantes hacia las funciones lógicas definidas en los controladores. Utiliza la clase `express.Router()` para estructurar una API REST modularizada por recursos.

- **usuarioRoutes.js**

Define los puntos de acceso para la autenticación, gestión de credenciales y consulta de perfiles de usuario

- Endpoints: Registro (POST /registro), Login (POST /login), Recuperación (POST /recuperar-password), Listado/Búsqueda (GET /, GET /buscar/:registro, GET /:id), Actualización (PUT /:id) y Cambio de contraseña (PUT /:id/password).

```
1  const express = require('express');
2
3  const {
4    registrarUsuario,
5    iniciarSesion,
6    obtenerUsuarios,
7    obtenerUsuarioPorId,
8    buscarPorRegistro,
9    actualizarPerfil,
10   cambiarPassword,
11   recuperarPassword
12 } = require('../controllers/usuarioController');
13
14 const router = express.Router();
15
16
17 // Registrar usuario
18 router.post('/registro', registrarUsuario);
19
20
21 // Iniciar sesión
22 router.post('/login', iniciarSesion);
23
24
25 // Recuperar contraseña
26 router.post('/recuperar-password', recuperarPassword);
27
28
29 // Obtener todos
30 router.get('/', obtenerUsuarios);
31
32
33 // Buscar por registro académico
34 router.get('/buscar/:registro', buscarPorRegistro);
35
36
37 // Obtener por ID
38 router.get('/:id', obtenerUsuarioPorId);
39
40
41 // Actualizar perfil
42 router.put('/:id', actualizarPerfil);
43
44
45 // Cambiar contraseña
46 router.put('/:id/password', cambiarPassword);
47
48
49 module.exports = router;
```

- **catedraticoRoutes.js**

Maneja las peticiones relacionadas con el cuerpo docente de la institución

- Endpoints: Búsqueda textual (GET /buscar/:texto), Listado general (GET /), Consulta individual (GET /:id) y Creación de catedráticos (POST /).

```

1  const express = require('express');
2
3  const {
4    obtenerCatedraticos,
5    obtenerCatedraticoPorId,
6    buscarCatedraticos,
7    crearCatedratico
8  } = require('../controllers/catedraticoController');
9
10 const router = express.Router();
11
12
13 // Buscar
14 router.get('/buscar/:texto', buscarCatedraticos);
15
16
17 // Todos
18 router.get('/', obtenerCatedraticos);
19
20
21 // Uno
22 router.get('/:id', obtenerCatedraticoPorId);
23
24
25 // Crear
26 router.post('/', crearCatedratico);
27
28
29 module.exports = router;

```

Rutas recurso Catedráticos

- **cursoRoutes.js**

Gestiona el catálogo de asignaturas disponibles en la plataforma

- Endpoints: Filtro por texto (GET /buscar/:texto), Obtención general (GET /), Consulta por ID (GET /:id) y Creación de cursos (POST /).

```

1  const express = require('express');
2
3  const {
4    obtenerCursos,
5    obtenerCursoPorId,
6    buscarCursos,
7    crearCurso
8  } = require('../controllers/cursoController');
9
10 const router = express.Router();
11
12
13 // Buscar cursos
14 router.get('/buscar/:texto', buscarCursos);
15
16
17 // Obtener todos
18 router.get('/', obtenerCursos);
19
20
21 // Obtener uno
22 router.get('/:id', obtenerCursoPorId);
23
24
25 // Crear curso
26 router.post('/', crearCurso);
27
28
29 module.exports = router;

```

Rutas recurso Cursos

- **publicacionRoutes.js**

Administra la creación y consulta de publicaciones o reseñas dentro del sistema

- Endpoints: Listado general con filtros (GET /), Creación (POST /), Consulta individual (GET /:id) y Eliminación de publicaciones (DELETE /:id).

```
1  const express = require('express');
2
3  const {
4      obtenerPublicaciones,
5      obtenerPublicacionPorId,
6      crearPublicacion,
7      eliminarPublicacion
8  } = require('../controllers/publicacionController');
9
10 const router = express.Router();
11
12
13 // Todas las publicaciones / filtros
14 router.get('/', obtenerPublicaciones);
15
16
17 // Crear publicación
18 router.post('/', crearPublicacion);
19
20
21 // Obtener una publicación
22 router.get('/:id', obtenerPublicacionPorId);
23
24
25 // Eliminar publicación
26 router.delete('/:id', eliminarPublicacion);
27
28
29 module.exports = router;
```

Rutas recurso Publicaciones

- **comentarioRoutes.js**

Controla la interacción y retroalimentación dentro de las publicaciones

- Endpoints: Obtención de comentarios por publicación (GET /publicacion/:id_publicacion), Creación (POST /) y Eliminación por ID (DELETE /:id).

```
1  const express = require('express');
2
3  const {
4      obtenerComentarios,
5      crearComentario,
6      eliminarComentario
7  } = require('../controllers/comentarioController');
8
9  const router = express.Router();
10
11
12  // Obtener comentarios de una publicación
13  router.get('/publicacion/:id_publicacion', obtenerComentarios);
14
15
16  // Crear comentario
17  router.post('/', crearComentario);
18
19
20  // Eliminar comentario
21  router.delete('/:id', eliminarComentario);
22
23
24  module.exports = router;
```

Rutas recurso Comentarios

- **cursoAprobadoRoutes.js**

Permite llevar el historial académico del usuario (pensum aprobado)

- Endpoints: Consulta de cursos por estudiante (GET /:id_usuario), Asignación/Aprobación de curso (POST /) y Eliminación de registro (DELETE /:id_usuario/:id_curso).

```
1  const express = require('express');
2
3  const {
4    obtenerCursosAprobados,
5    agregarCursoAprobado,
6    eliminarCursoAprobado
7  } = require('../controllers/cursoAprobadoController');
8
9  const router = express.Router();
10
11
12  // Obtener cursos aprobados de un usuario
13  router.get('/:id_usuario', obtenerCursosAprobados);
14
15
16  // Agregar curso aprobado
17  router.post('/', agregarCursoAprobado);
18
19
20  // Eliminar curso aprobado
21  router.delete(
22    '/:id_usuario/:id_curso',
23    eliminarCursoAprobado
24  );
25
26
27  module.exports = router;
```

Rutas recurso Cursos Aprobados

src/ controllers.js

Controllers contiene la lógica de negocio de la aplicación. Su función principal es recibir las peticiones HTTP procesadas por las rutas, interactuar con la base de datos MySQL a través del pool de conexiones, validar los datos de entrada y retornar las respuestas en formato JSON con sus respectivos códigos de estado HTTP.

1. src/controllers/usuarioController.js

Este controlador gestiona el registro de usuarios, autenticación y la edición de información del perfil académico:

- **Gestión de Seguridad de Contraseñas (generarPassword y verificarPassword):** Implementa el módulo nativo **crypto** mediante el algoritmo **scryptSync** combinado con una sal (salt) aleatoria de 16 bytes. Para la autenticación utiliza **crypto.timingSafeEqual**, protegiendo el sistema contra ataques de tiempos (timing attacks).

- **Registro de Usuarios (registrarUsuario):** Verifica la no existencia previa del Registro Personal o correo en MySQL. Genera el hash con sal y almacena al usuario retornando el código HTTP 201 (Created).
- **Inicio de Sesión (iniciarSesion):** Autentica al estudiante mediante su registro académico y contraseña. Devuelve los datos básicos del perfil tras comparar el hash criptográfico.
- **Búsqueda y Listado (obtenerUsuarios, obtenerUsuarioPorId, buscarPorRegistro):** Permite consultar la nómina de estudiantes, obtener detalles por ID y realizar búsquedas parciales por número de registro usando el operador **LIKE**.
- **Edición y Recuperación (actualizarPerfil, cambiarPassword, recuperarPassword):** Ofrece funciones para actualizar datos personales validando que el correo no esté ocupado por otro usuario, modificar la clave actual y restablecer credenciales en caso de olvido.

```

145 // =====
146 // INICIAR SESIÓN
147 // =====
148
149 ▼ const iniciarSesion = async (req, res) => {
150
151     try {
152
153         const {
154             registro_academico,
155             password
156         } = req.body;
157
158
159         if (!registro_academico || !password) {
160
161             return res.status(400).json({
162                 mensaje: 'El registro académico y la contraseña son obligatorios'
163             });
164
165         }
166
167
168         const [usuarios] = await pool.query(
169             `SELECT *
170              FROM usuarios
171              WHERE registro_academico = ?`,
172             [registro_academico]
173         );

```

```

174
175
176     if (usuarios.length === 0) {
177
178         return res.status(401).json({
179             mensaje: 'Registro académico o contraseña incorrectos'
180         });
181
182     }
183
184
185     const usuario = usuarios[0];
186
187
188     const passwordCorrecta = verificarPassword(
189         password,
190         usuario.password
191     );
192
193
194     if (!passwordCorrecta) {
195
196         return res.status(401).json({
197             mensaje: 'Registro académico o contraseña incorrectos'
198         });
199
200     }
201
202
203     res.json({
204
205         mensaje: 'Inicio de sesión exitoso',
206
207         usuario: {
208             id_usuario: usuario.id_usuario,
209             registro_academico: usuario.registro_academico,
210             nombres: usuario.nombres,
211             apellidos: usuario.apellidos,
212             correo: usuario.correo
213         }
214
215     });
216
217
218     } catch (error) {
219
220         console.error('Error al iniciar sesión:', error);
221
222         res.status(500).json({
223             mensaje: 'Error al iniciar sesión',
224             error: error.message
225         });
226
227     }
228 };
229

```


2. src/controllers/publicacionController.js

Encargado de la gestión de publicaciones e interacciones principales del foro:

- Consulta Dinámica (obtenerPublicaciones): Ejecuta consultas complejas con INNER JOIN hacia usuarios y LEFT JOIN hacia cursos y catedráticos. Permite filtrar los resultados de forma dinámica por id_curso, id_catedratico o búsquedas de texto con LIKE.
- Consulta por ID (obtenerPublicacionPorId): Recupera la información detallada de un post específico combinando nombres de autores, cursos o catedráticos vinculados.
- Creación de Publicación (crearPublicacion): Aplica reglas de negocio donde una publicación debe asociarse obligatoriamente a un curso o a un catedrático, pero nunca a ambos al mismo tiempo.
- Eliminación (eliminarPublicacion): Borra la publicación objetivo de la base de datos validando su existencia mediante las filas afectadas (affectedRows).

```
135 // =====
136 // OBTENER PUBLICACIÓN POR ID
137 // =====
138
139 ▼ const obtenerPublicacionPorId = async (req, res) => {
140
141     try {
142
143         const { id } = req.params;
144
145
146         const [publicaciones] = await pool.query(
147             `
148             SELECT
149                 p.id_publicacion,
150                 p.id_usuario,
151                 CONCAT(u.nombres, ' ', u.apellidos) AS usuario,
152
153                 p.id_curso,
154                 c.codigo AS codigo_curso,
155                 c.nombre AS nombre_curso,
156
157                 p.id_catedratico,
158                 CONCAT(cat.nombres, ' ', cat.apellidos) AS catedratico,
159
160                 p.mensaje,
161                 p.fecha_creacion
162
163             FROM publicaciones p
```

```

164
165         INNER JOIN usuarios u
166             ON p.id_usuario = u.id_usuario
167
168         LEFT JOIN cursos c
169             ON p.id_curso = c.id_curso
170
171         LEFT JOIN catedraticos cat
172             ON p.id_catedratico = cat.id_catedratico
173
174         WHERE p.id_publicacion = ?
175         `,
176         [id]
177     );
178
179
180     if (publicaciones.length === 0) {
181
182         return res.status(404).json({
183             mensaje: 'Publicación no encontrada'
184         });
185
186     }
187
188
189     res.json(publicaciones[0]);
190
191
192     } catch (error) {
193
194         console.error(
195             'Error al obtener publicación:',
196             error
197         );
198
199         res.status(500).json({
200             mensaje: 'Error al obtener publicación',
201             error: error.message
202         });
203
204     }
205 };
206
207

```

3. src/controllers/cursoAprobadoController.js

Administra el historial académico del estudiante:

- Consulta de Avance (obtenerCursosAprobados): Realiza un INNER JOIN entre las tablas cursos_aprobados y cursos para obtener el listado de materias aprobadas por el estudiante y calcula dinámicamente la suma acumulada de créditos obtenidos mediante la función .reduce().
- Registro y Eliminación (agregarCursoAprobado, eliminarCursoAprobado): Permite asignar asignaturas al historial previniendo registros duplicados (409 Conflict) y remover materias del historial académico según el id_usuario e id_curso.

```
4 // =====
5 // OBTENER CURSOS APROBADOS DE UN USUARIO
6 // =====
7
8 const obtenerCursosAprobados = async (req, res) => {
9
10   try {
11
12     const { id_usuario } = req.params;
13
14
15     const [cursos] = await pool.query(
16
17       SELECT
18         c.id_curso,
19         c.codigo,
20         c.nombre,
21         c.creditos,
22         ca.fecha_aprobacion
23
24       FROM cursos_aprobados ca
25
26       INNER JOIN cursos c
27         ON ca.id_curso = c.id_curso
28
29       WHERE ca.id_usuario = ?
30
31       ORDER BY c.codigo
32       ,
33       [id_usuario]
34     );
35
36
37     const totalCreditos = cursos.reduce(
38       (total, curso) => total + Number(curso.creditos),
39       0
40     );
41
42
43     res.json({
44
45       cursos: cursos,
46
47       totalCreditos: totalCreditos
48
49     });
50
51   } catch (error) {
52
53     console.error(
54       "Error al obtener cursos aprobados:",
55       error
56     );
57
58
59     res.status(500).json({
60       mensaje: "Error al obtener cursos aprobados",
61       error: error.message
62     });
63
64   }
65 }
```

4. src/controllers/comentarioController.js

Gestiona el hilo de respuestas en cada publicación:

- Obtener Comentarios (obtenerComentarios): Recupera en orden cronológico ascendente (ASC) los comentarios pertenecientes a una publicación, incluyendo el nombre completo y registro del emisor.
- Creación y Eliminación (crearComentario, eliminarComentario): Registra la interacción en la tabla comentarios tras validar los campos obligatorios y gestiona la remoción de comentarios específicos por ID.

```
5 // OBTENER COMENTARIOS DE UNA PUBLICACIÓN
6 // =====
7
8 const obtenerComentarios = async (req, res) => {
9
10   try {
11
12     const { id_publicacion } = req.params;
13
14
15     const [comentarios] = await pool.query(
16       `
17       SELECT
18         co.id_comentario,
19         co.id_publicacion,
20         co.id_usuario,
21
22         CONCAT(
23           u.nombres,
24           ' ',
25           u.apellidos
26         ) AS usuario,
27
28         u.registro_academico,
29
30         co.mensaje,
31         co.fecha_creacion
32
33       FROM comentarios co
34
35       INNER JOIN usuarios u
36         ON co.id_usuario = u.id_usuario
37
38       WHERE co.id_publicacion = ?
39
40       ORDER BY co.fecha_creacion ASC
41       ,
42       [id_publicacion]
43     `);
44
45     res.json(comentarios);
46
47   } catch (error) {
48
49     console.error(
50       'Error al obtener comentarios:',
51       error
52     );
53
54     res.status(500).json({
55       mensaje: 'Error al obtener comentarios',
56       error: error.message
57     });
58
59   }
60 }
```

5. src/controllers/cursoController.js

Ofrece la lectura del pensum de la carrera, filtro por código o nombre de materia mediante LIKE, y registro de nuevas asignaturas con sus respectivos créditos (obtenerCursos, obtenerCursoPorId, buscarCursos, crearCurso).

```
4 // =====
5 // OBTENER TODOS LOS CURSOS
6 // =====
7
8 const obtenerCursos = async (req, res) => {
9
10   try {
11
12     const [cursos] = await pool.query(
13       `SELECT
14         id_curso,
15         codigo,
16         nombre,
17         creditos
18       FROM cursos
19       ORDER BY codigo`
20     );
21
22     res.json(cursos);
23
24   } catch (error) {
25
26     console.error('Error al obtener cursos:', error);
27
28     res.status(500).json({
29       mensaje: 'Error al obtener cursos',
30       error: error.message
31     });
32
33   }
34 }
35
36 ;
```

6. src/controllers/catedraticoController.js

Permite consultar la lista de profesores ordenada alfabéticamente por apellido, realizar búsquedas dinámicas por nombre/apellido y dar de alta a nuevos catedráticos en la base de datos (obtenerCatedraticos, buscarCatedraticos, crearCatedratico).

```
85 // =====
86 // BUSCAR CATEDRÁTICOS
87 // =====
88
89 const buscarCatedraticos = async (req, res) => {
90
91   try {
92
93     const { texto } = req.params;
94
95     const [catedraticos] = await pool.query(
96       `SELECT
97         id_catedratico,
98         nombres,
99         apellidos
100      FROM catedraticos
101      WHERE nombres LIKE ?
102      OR apellidos LIKE ?
103      ORDER BY apellidos, nombres`,
104       [
105         `%${texto}%`,
106         `%${texto}%`
107       ]
108     );
109
110     res.json(catedraticos);
111
112   }
113 }
```

```
114
115 } catch (error) {
116
117   console.error('Error al buscar catedráticos:', error);
118
119   res.status(500).json({
120     mensaje: 'Error al buscar catedráticos',
121     error: error.message
122   });
123
124 }
125 ;
```

/.env

Este archivo contiene las variables de entorno para la configuración del servidor y la conexión a la base de datos MySQL. Permite parametrizar credenciales sensibles y puertos sin exponerlos directamente en el código fuente.

```
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD=root
DB_NAME=practica_desarrollo_web
PORT=3000
```

/package.json

Este archivo define la configuración principal del proyecto Node.js. Especifica el punto de entrada (src/app.js), los scripts de ejecución (npm run dev / npm start) y administra las dependencias necesarias como Express, MySQL2, CORS y Dotenv.

```
{
  "name": "backend",
  "version": "1.0.0",
  "description": "Backend REST API para la actividad de Desarrollo Web - Practicas Iniciales",
  "main": "src/app.js",
  "scripts": {
    "start": "node src/app.js",
    "dev": "nodemon src/app.js"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "cors": "^2.8.6",
    "dotenv": "^17.4.2",
    "express": "^5.2.1",
    "mysql2": "^3.24.4"
  },
  "devDependencies": {
    "nodemon": "^3.1.14"
  }
}
```

/package-lock.json

Garantiza la instalación consistente y reproducible de las dependencias del proyecto, bloqueando las versiones exactas de cada paquete instalado y sus dependencias transitivas.

/node_modules

Esta carpeta se genera automáticamente al ejecutar npm install. Almacena todo el código ejecutable de las dependencias externas y paquetes de Node.js (como Express, MySQL2, Dotenv, CORS) necesarios para que el backend funcione localmente.

FRONTEND

CLASES Y MODULOS UTILIZADOS

/Public

Almacena los recursos estáticos públicos del proyecto a los que el navegador puede acceder directamente sin pasar por el proceso de compilación de React.

- **favicon.svg:** Icono vectorial que se muestra en la pestaña del navegador junto al título de la aplicación.
- **icons.svg:** Sprite gráfico en formato SVG que reúne múltiples íconos vectoriales reutilizables dentro de la interfaz de usuario.

/Src

src/components

- Navbar.jsx: Componente de barra de navegación global. Maneja la redirección entre las secciones principales (Inicio, Crear Publicación, Mi Perfil) mediante React Router y gestiona la funcionalidad de cierre de sesión invocando el contexto de autenticación.

```
import { Link, useNavigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';

function Navbar() {
  const { usuario, logout } = useAuth();
  const navigate = useNavigate();

  const handleLogout = () => {
    logout();
    navigate('/');
  };

  return (
    <nav className="navbar">
      <div className="navbar-brand">
        <Link to="/inicio">ECYS - R cord Catedr ticos</Link>
      </div>
      <div className="navbar-links">
        <Link to="/inicio">Inicio</Link>
        <Link to="/crear-publicacion">Crear Publicaci n</Link>
        <Link to={` /perfil/${usuario?.id_usuario}`}>Mi Perfil</Link>
        <button onClick={handleLogout}>Cerrar sesi n</button>
      </div>
    </nav>
  );
}

export default Navbar;
```

- PublicacionCard.jsx: Componente modular para renderizar las publicaciones individuales. Muestra detalles del autor, etiquetas de curso o catedr tico y la fecha en formato local. Incluye l gica din mica para desplegar, cargar y enviar comentarios consumiendo la API backend en tiempo real.

```
import { useState, useEffect } from 'react';
import api from '../services/api';
import { useAuth } from '../context/AuthContext';

function PublicacionCard({ publicacion }) {
  const { usuario } = useAuth();
  const [comentarios, setComentarios] = useState([]);
  const [mostrarComentarios, setMostrarComentarios] = useState(false);
  const [nuevoComentario, setNuevoComentario] = useState('');
  const [cargando, setCargando] = useState(false);

  const cargarComentarios = async () => {
    try {
      const res = await api.get(`/comentarios/publicacion/${publicacion.id_publicacion}`);
      setComentarios(res.data);
    } catch (err) {
      console.error('Error al cargar comentarios:', err);
    }
  };

  const toggleComentarios = async () => {
    if (!mostrarComentarios) {
      await cargarComentarios();
    }
    setMostrarComentarios(!mostrarComentarios);
  };

  const enviarComentario = async (e) => {
    e.preventDefault();
    if (!nuevoComentario.trim()) return;
    setCargando(true);

    try {
      await api.post('/comentarios', {
        id_publicacion: publicacion.id_publicacion,
        id_usuario: usuario.id_usuario,
        mensaje: nuevoComentario,
      });
      setNuevoComentario('');
      await cargarComentarios();
    } catch (err) {
      console.error('Error al enviar comentario:', err);
    }
  };
}
```



```

    await cargarComentarios();
  } catch (err) {
    console.error('Error al enviar comentario:', err);
  } finally {
    setCargando(false);
  }
};

```

src/context

- **AuthContext.jsx:** Proveedor de contexto global de autenticación en React. Administra el estado de la sesión del usuario activo (**usuario**) y persiste sus datos en **localStorage**. Ofrece funciones auxiliares (**login**, **logout**) y el hook personalizado **useAuth()** para compartir la información de sesión entre todas las páginas y componentes de la aplicación.

```

import { createContext, useContext, useState, useEffect } from 'react';

const AuthContext = createContext();

export const AuthProvider = ({ children }) => {
  const [usuario, setUsuario] = useState(null);

  useEffect(() => {
    const guardado = localStorage.getItem('usuario');
    if (guardado) {
      setUsuario(JSON.parse(guardado));
    }
  }, []);

  const login = (datosUsuario) => {
    setUsuario(datosUsuario);
    localStorage.setItem('usuario', JSON.stringify(datosUsuario));
  };

  const logout = () => {
    setUsuario(null);
    localStorage.removeItem('usuario');
  };

  return (
    <AuthContext.Provider value={{ usuario, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
};

export const useAuth = () => useContext(AuthContext);

```

src/pages/

- **Login.jsx:** Vista de inicio de sesión para los estudiantes. Proporciona un formulario para validar las credenciales (**registro_academico** y **password**) contra la API, almacena el usuario autenticado en el contexto global y redirige hacia la vista principal (**/inicio**). Incluye enlaces de navegación hacia la creación de cuenta y recuperación de contraseña.

```

<form onSubmit={handleSubmit}>
  <input
    type="text"
    placeholder="CUI / REGISTRO ACADÉMICO / REGISTRO PERSONAL"
    value={registro}
    onChange={(e) => setRegistro(e.target.value)}
    required
  />

  <input
    type="password"
    placeholder="Contraseña"
    value={password}
    onChange={(e) => setPassword(e.target.value)}
    required
  />

  <label>
    <input type="checkbox" /> Recordar mi usuario
  </label>

  {error && <p className="error">{error}</p>}

  <button type="submit" disabled={cargando}>
    {cargando ? 'Iniciando...' : 'INICIAR SESIÓN'}
  </button>
</form>

```


- **Registro.jsx:** Pantalla de creación de cuenta. Recopila los datos personales obligatorios (registro académico, nombres, apellidos, correo y contraseña) mediante un estado **form** unificado y consume el endpoint **POST /usuarios/registro** para registrar nuevos usuarios en el sistema.

```
const handleSubmit = async (e) => {
  e.preventDefault();
  setError('');
  setCargando(true);

  try {
    await api.post('/usuarios/registro', form);
    alert('Usuario registrado correctamente');
    navigate('/');
  } catch (err) {
    setError(err.response?.data?.mensaje || 'Error al registrar');
  } finally {
    setCargando(false);
  }
};
```

- **RecuperarPassword.jsx:** Vista para el restablecimiento de credenciales de acceso. Solicita la confirmación de la identidad del usuario a través de su registro académico y correo electrónico, permitiendo ingresar una nueva contraseña para ser enviada al backend mediante **POST /usuarios/recuperar-password**.

```
const handleSubmit = async (e) => {
  e.preventDefault();
  setError('');
  setCargando(true);

  try {
    await api.post('/usuarios/recuperar-password', form);
    alert('Contraseña restablecida correctamente');
    navigate('/');
  } catch (err) {
    setError(err.response?.data?.mensaje || 'Datos incorrectos');
  } finally {
    setCargando(false);
  }
};
```

- **Inicio.jsx:** Pantalla principal del feed de la plataforma. Muestra el listado dinámico de publicaciones con capacidad de búsqueda y filtrado combinado (por ID de curso, ID de catedrático, o búsqueda textual de nombre de curso/catedrático) implementando un debounce de 300 ms. Adicionalmente, incluye un buscador para redirigir a la perfilación de otros estudiantes.

```
// Cargar publicaciones cada vez que cambia algún filtro
useEffect(() => {
  const cargarPublicaciones = async () => {
    setCargando(true);
    try {
      const params = {};
      if (filtroCursoId) params.id_curso = filtroCursoId;
      if (filtroCatedraticoId) params.id_catedratico = filtroCatedraticoId;
      if (filtroNombreCurso) params.curso = filtroNombreCurso;
      if (filtroNombreCatedratico) params.profesor = filtroNombreCatedratico;

      const res = await api.get('/publicaciones', { params });
      setPublicaciones(res.data);
    } catch (err) {
      console.error('Error al cargar publicaciones:', err);
    } finally {
      setCargando(false);
    }
  };

  const timeout = setTimeout(cargarPublicaciones, 300);
  return () => clearTimeout(timeout);
}, [filtroCursoId, filtroCatedraticoId, filtroNombreCurso, filtroNombreCatedratico]);
```

- **CrearPublicacion.jsx:** Interfaz de creación de comentarios o reseñas. Permite seleccionar dinámicamente si la publicación estará vinculada a un curso o a un catedrático mediante botones de opción (radio buttons), cargando previamente los listados auxiliares desde la API y enviando la publicación a través de **POST /publicaciones**.

```
<div className="tipo-selector">
  <label>
    <input
      type="radio"
      value="curso"
      checked={tipo === 'curso'}
      onChange={(e) => setTipo(e.target.value)}
    />
    Sobre un Curso
  </label>
  <label>
    <input
      type="radio"
      value="catedratico"
      checked={tipo === 'catedratico'}
      onChange={(e) => setTipo(e.target.value)}
    />
    Sobre un Catedrático
  </label>
</div>
```

sección del selector de tipo tipo-selector en el JSX

- **Perfil.jsx:** Vista detallada del perfil de usuario. Renderiza la información académica del estudiante y detecta automáticamente si se trata del usuario logueado (**esMiPerfil**) para habilitar la edición interactiva de sus datos (nombres, apellidos y correo) utilizando **PUT /usuarios/:id**.

```
// Validación para verificar si la vista corresponde al usuario autenticado
const esMiPerfil = Number(id) === Number(usuarioLogueado?.id_usuario);

// Handler para la actualización de datos personales mediante la API
const handleGuardar = async (e) => {
  e.preventDefault();
  setGuardando(true);
  try {
    await api.put(`/usuarios/${id}`, form);
    setPerfil({ ...perfil, ...form });
    setEditando(false);
  } catch (err) {
    alert(err.response?.data?.mensaje || 'Error al actualizar');
  } finally {
    setGuardando(false);
  }
};
```

- **BuscarUsuario.jsx:** Vista de resultados de búsqueda de estudiantes por registro académico. Consulta el endpoint **GET /usuarios/buscar/:registro** a partir de los parámetros de la URL (**useParams**) y presenta un listado de tarjetas con acceso directo a los perfiles encontrados.

```
<div className="resultados-usuarios">
  {resultados.map((u) => (
    <div key={u.id_usuario} className="usuario-card">
      <div>
        <strong>{u.nombres} {u.apellidos}</strong>
        <p className="registro">Registro: {u.registro_academico}</p>
        <p className="correo">{u.correo}</p>
      </div>
      <Link to={`/perfil/${u.id_usuario}`} className="btn-secundario">
        Ver perfil
      </Link>
    </div>
  ))}
</div>
```

resultados.map(...)

- **CursosAprobados.jsx:** Gestión de avance académico del estudiante. Presenta una tabla con la lista de cursos aprobados y la suma total de créditos acumulados. Si el usuario está viendo su propio perfil, la pantalla despliega controles interactivos para agregar nuevos cursos aprobados o eliminar existentes mediante llamados a la API.

```
const agregarCurso = async (e) => {
  e.preventDefault();
  if (!cursoSeleccionado) return;

  try {
    await api.post('/cursos-aprobados', {
      id_usuario: Number(id),
      id_curso: Number(cursoSeleccionado),
    });
    setCursoSeleccionado('');
    await cargarDatos();
  } catch (err) {
    alert(err.response?.data?.mensaje || 'Error al agregar curso');
  }
};

const eliminarCurso = async (idCurso) => {
  if (!window.confirm('¿Eliminar este curso de aprobados?')) return;
  try {
    await api.delete(`/cursos-aprobados/${id}/${idCurso}`);
    await cargarDatos();
  } catch (err) {
    alert(err.response?.data?.mensaje || 'Error al eliminar');
  }
};

// Cursos disponibles (los que no están aprobados aún)
const cursosDisponibles = todosCursos.filter(
  (c) => !cursosAprobados.some((ca) => ca.id_curso === c.id_curso)
);
```

src/services/

- **api.js:** Módulo centralizado para la configuración del cliente HTTP. Utiliza la librería Axios para crear una instancia personalizada (**api**) que define la URL base del servidor backend (**http://localhost:3000/api**) y los encabezados globales para las peticiones JSON. Esto evita repetir la dirección del servidor en cada una de las pantallas de la aplicación.

```
import axios from 'axios';

const api = axios.create({
  baseUrl: 'http://localhost:3000/api',
  headers: {
    'Content-Type': 'application/json',
  },
});

export default api;
```

src/styles/

Directorio de Hoja de Estilos Complementaria: Estructura destinada a albergar archivos **.css** o módulos de diseño independientes (actualmente reservada para escalabilidad futura del diseño modular).

src/ App.css

Hoja de estilos global de la aplicación frontend. Define las reglas CSS estructuradas por módulos funcionales: reseteo CSS básico, variables visuales del tema (paleta azul institucional **#1a3d7c**), diseño del navegador (**navbar**), maquetación de tarjetas, tablas de avance académico y consultas de medios.

```
/* =====  
RESET Y BASE  
===== */  
  
* {  
  margin: 0;  
  padding: 0;  
  box-sizing: border-box;  
}  
  
body {  
  font-family: Arial, Helvetica, sans-serif;  
  background: #f0f2f5;  
  color: #333;  
}
```

```
/* =====  
LAYOUT DE LA APP  
===== */  
  
.app-container {  
  min-height: 100vh;  
  background: #f0f2f5;  
}  
  
.main-content {  
  max-width: 900px;  
  margin: 0 auto;  
  padding: 24px;  
}  
  
.main-content h1 {  
  color: #1a3d7c;  
  margin-bottom: 20px;  
}
```

```
/* =====  
NAVBAR  
===== */  
  
.navbar {  
  background: #1a3d7c;  
  color: white;  
  padding: 14px 24px;  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  box-shadow: 0 2px 6px rgba(0, 0, 0, 0.15);  
}  
  
.navbar-brand a {  
  color: white;  
  font-weight: bold;  
  text-decoration: none;  
  font-size: 16px;  
}
```

src/ App.jsx

Componente raíz de la aplicación frontend. Se encarga de envolver el sistema dentro del proveedor global de sesión (**AuthProvider**) y configurar el mapa de rutas con **React Router DOM**. Implementa un componente de alto orden llamado **RutaProtegida** para restringir el acceso a usuarios no autenticados, redirigiéndolos al inicio de sesión si intentan navegar a vistas privadas.

```

import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom';
import { AuthProvider, useAuth } from '../context/AuthContext';
import Login from '../pages/Login';
import Registro from '../pages/Registro';
import RecuperarPassword from '../pages/RecuperarPassword';
import Inicio from '../pages/Inicio';
import CrearPublicacion from '../pages/CrearPublicacion';
import Perfil from '../pages/Perfil';
import CursosAprobados from '../pages/CursosAprobados';
import BuscarUsuario from '../pages/BuscarUsuario';
import './App.css';

function RutaProtegida({ children }) {
  const { usuario } = useAuth();
  return usuario ? children : <Navigate to="/" />;
}

function App() {
  return (
    <AuthProvider>
      <BrowserRouter>
        <Routes>
          <Route path="/" element={<Login />} />
          <Route path="/registro" element={<Registro />} />
          <Route path="/recuperar" element={<RecuperarPassword />} />
          <Route path="/inicio" element={<RutaProtegida><Inicio /></RutaProtegida>} />
          <Route path="/crear-publicacion" element={<RutaProtegida><CrearPublicacion /></RutaProtegida>} />
          <Route path="/perfil/:id" element={<RutaProtegida><Perfil /></RutaProtegida>} />
          <Route path="/perfil/:id/cursos" element={<RutaProtegida><CursosAprobados /></RutaProtegida>} />
          <Route path="/buscar/:registro" element={<RutaProtegida><BuscarUsuario /></RutaProtegida>} />
        </Routes>
      </BrowserRouter>
    </AuthProvider>
  );
}

export default App;

```

src/ index.css

Archivo de estilos globales de nivel superior. En este proyecto se encuentra en blanco debido a que la totalidad del diseño técnico visual está centralizado en **App.css**.

src/ main.jsx

Punto de entrada principal de la aplicación React. Se encarga de inicializar el árbol de componentes del DOM virtual mediante la API **createRoot** de React 18+, montando el componente raíz **<App/>** dentro del nodo contenedor HTML con ID **'root'**. Incluye la envoltura **<StrictMode>** para activar advertencias y comprobaciones adicionales en el entorno de desarrollo.

```

import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)

```

/index.html

Estructura base del documento HTML5 de la aplicación Single Page Application (SPA). Contiene la etiqueta contenedora `<div id="root"></div>` donde se renderiza la aplicación de React y la llamada principal al script `src/main.jsx`.

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>frontend</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

/.gitignore

Archivo de configuración del sistema de control de versiones Git. Define las reglas de exclusión para omitir archivos y carpetas que no deben subirse al repositorio remoto, tales como dependencias instaladas (**node_modules**), la carpeta de compilación final (**dist/**), archivos de registro de errores (***.log**) y configuraciones de editores (**.vscode/**).

/.oxlintrc.json

Archivo de configuración del linter estático Oxlint. Establece las reglas de análisis de código para React, asegurando el cumplimiento estricto de las reglas de los Hooks (**rules-of-hooks**) y advirtiendo sobre malas prácticas en la exportación de componentes.

```
{
  "$schema": "./node_modules/oxlint/configuration_schema.json",
  "plugins": ["react", "oxc"],
  "rules": {
    "react/rules-of-hooks": "error",
    "react/only-export-components": ["warn", { "allowConstantExport": true }]
  }
}
```

/package.json

Manifiesto principal del proyecto Frontend. Define los metadatos de la aplicación, los comandos de ejecución (**scripts**) para desarrollo, compilación y validación, así como las dependencias de producción (**axios, react, react-router-dom**) y desarrollo (**vite, oxlint**).

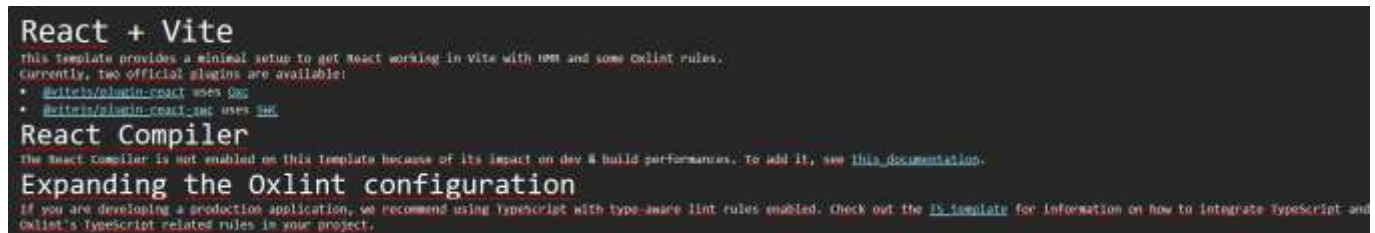
```
{
  "name": "frontend",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "oxlint",
    "preview": "vite preview"
  },
  "dependencies": {
    "axios": "^1.20.0",
    "react": "^19.2.8",
    "react-dom": "^19.2.8",
    "react-router-dom": "^7.18.4"
  },
  "devDependencies": {
    "@types/react": "^19.2.18",
    "@types/react-dom": "^19.2.7",
    "@vitejs/plugin-react": "^6.1.1",
    "oxlint": "^1.81.0",
    "vite": "^8.3.0"
  }
}
```


/package-lock.json

Archivo de registro generado automáticamente por Node Package Manager (npm). Asegura la reproducibilidad de las instalaciones al bloquear las versiones exactas de todas las dependencias y subdependencias instaladas en el árbol del proyecto.

/README.md

Documento explicativo de bienvenida autogenerado por la plantilla base de Vite. Contiene información básica sobre el entorno de desarrollo con React, el servidor local HMR (Hot Module Replacement) y el uso de las reglas de linting con Oxlint.



/vite.config.js

Archivo de configuración principal de **Vite**. Define los complementos (plugins) necesarios para el procesamiento del proyecto, activando el plugin oficial de React (**@vitejs/plugin-react**) para habilitar la compilación rápida de sintaxis JSX y la recarga en tiempo real (Fast Refresh / HMR) durante el desarrollo.

```
import react from '@vitejs/plugin-react'
import { defineConfig } from 'vite'

// https://vite.dev/config/
export default defineConfig({
  plugins: [react()],
})
```

/Node_modules

La carpeta node_modules se genera automáticamente al ejecutar comandos como npm i -D @vitejs/plugin-react, donde -D indica que el paquete se instala como dependencia de desarrollo. Esta carpeta almacena todos los paquetes y plugins descargados, en este caso, el plugin de React para Vite, permitiendo que el proyecto los use sin tener que instalarlos manualmente cada vez.

GUÍA DE EJECUCIÓN Y DESPLIEGUE DEL SISTEMA

- **PASO 1: CONFIGURACIÓN DE LA BASE DE DATOS (MySQL)**
 1. Iniciar el servicio de MySQL local (vía MySQL Workbench, XAMPP o Servicio del Sistema)
Confirmar que el servidor MySQL se encuentre escuchando en el puerto estándar 3306.
 2. Cargar el esquema e información inicial
 - Abrir MySQL Workbench (o CLI) y ejecutar el archivo de inicialización:
Ubicación: database/script.sql
 - Este script creará automáticamente la base de datos 'practica_desarrollo_web', las 6 tablas principales y los datos de prueba iniciales.

- **PASO 2: CONFIGURACIÓN Y EJECUCIÓN DEL BACKEND (Node.js + Express)**
 - 3. Navegar a la carpeta del backend desde la raíz del proyecto
cd backend
 - 4. Crear el archivo de variables de entorno (.env) en la raíz de 'backend/'
Debe contener las siguientes credenciales de conexión:
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD=root
DB_NAME=practica_desarrollo_web
PORT=3000
 - 5. Instalar las dependencias de Node.js (express, mysql2, cors, dotenv, etc.)
npm install
 - 6. Iniciar el servidor backend en modo desarrollo con Nodemon
npm run dev
El servidor estará escuchando en http://localhost:3000
 - 7. Verificar el estado de la conexión con la base de datos
Probar la API desde el navegador o cliente HTTP: http://localhost:3000/api/prueba-db
- **PASO 3: CONFIGURACIÓN Y EJECUCIÓN DEL FRONTEND (React + Vite)**
 - 8. Navegar a la carpeta del frontend
cd ../frontend
 - 9. Instalar las dependencias de React (axios, react-router-dom, etc.)
npm install
 - 10. Iniciar el servidor de desarrollo Vite
npm run dev
 - 11. Abrir la aplicación en el navegador web
Acceder a la URL local asignada por Vite: http://localhost:5173/

3. SECCIÓN DE AYUDA Y RESOLUCIÓN DE PROBLEMAS (TROUBLESHOOTING)

A continuación, se presentan los errores más comunes que pueden ocurrir durante el uso de la aplicación web y su respectiva solución:

Incidencia Detectada	Análisis y Causas Posibles	Solución Aplicada
Error ECONNREFUSED 127.0.0.1:3306 al iniciar el backend	El 1. El servicio local de MySQL está detenido. 2. Las credenciales en el archivo .env (DB_USER, DB_PASSWORD o DB_PORT) son incorrectas.	1. Iniciar el servicio de MySQL desde MySQL Workbench o la consola de servicios del sistema. 2. Verificar y corregir las variables de conexión en backend/.env.
Error de CORS (Cross-Origin Request Blocked) en la consola del navegador	El backend no está permitiendo peticiones provenientes del origen del frontend (http://localhost:5173).	Asegurar que el middleware cors() esté correctamente importado e inicializado en el archivo principal del servidor (src/app.js).
Página en blanco o error 404 al recargar rutas en el Frontend	react-router-dom no encuentra la ruta servida directamente por el servidor de desarrollo al refrescar.	Verificar las rutas configuradas en el Router del frontend y asegurar que el servidor de desarrollo de Vite redirija correctamente los fallbacks a index.html.
Error ER_NO_SUCH_TABLE al realizar peticiones a la API	Las peticiones al backend intentan consultar tablas que no han sido creadas en la base de datos.	Ejecutar nuevamente el script database/script.sql en MySQL Workbench para garantizar que la base de datos practica_desarrollo_web contenga todo el esquema y los datos iniciales.

Para más consultas a continuación te brindamos algunas opciones que podrían ser de utilidad:

- MySQL Documentation Hub: Manual técnico para la creación de esquemas de bases de datos, sintaxis consultas SQL y optimización de tablas relacionales.
- GitHub Docs: Repositorio de soporte para la gestión de repositorios en línea, solución de conflictos en commits y trabajo colaborativo en ramas.
- Node.js & Express Documentation: Guías oficiales para la creación de servicios REST API, manejo de middlewares y gestión de paquetes con NPM.

CONCLUSIONES

- Eficiencia en la Arquitectura Desacoplada y Escalabilidad: La implementación de una arquitectura Cliente Servidor desacoplada mediante un servicio REST API en Node.js y Express demuestra ser una solución óptima para el procesamiento asíncrono de peticiones HTTP. Esta separación entre la lógica de negocio y la interfaz de usuario no solo optimiza el rendimiento del servidor, sino que también permite escalar o reemplazar la capa del cliente (ReactJS o Angular) sin alterar el funcionamiento interno ni los contratos de endpoint de la aplicación.
- Integridad y Normalización en el Modelo de Datos Relacional: El diseño del esquema de base de datos en MySQL garantiza la consistencia e integridad referencial de la información mediante el uso estratégico de claves primarias y foráneas. La estructura relacional permite asociar eficazmente las opiniones y comentarios a los registros académicos de los estudiantes y a los catálogos de catedráticos o cursos, previniendo la duplicidad de datos y facilitando consultas complejas para los filtros de búsqueda del feed principal.
- Seguridad y Control de Acceso en el Servidor: La integración de mecanismos de validación de credenciales, encriptado de contraseñas y middleware para la gestión de CORS asegura que la API REST responda de forma segura únicamente a peticiones autorizadas. Esto previene accesos no permitidos a las rutas de actualización de perfil o asignación de cursos aprobados, resguardando la información personal del estudiante dentro del sistema.

REFERENCIAS

- Node.js Documentation: <https://nodejs.org/docs/>
- Express.js API Reference: <https://expressjs.com/>
- Vite Documentation: <https://vite.dev/>
- React Documentation: <https://react.dev/>
- MySQL 8.0 Reference Manual: <https://dev.mysql.com/doc/refman/8.0/en/>
- Pensum y Cursos ECYS (DTT): <https://dtc-ecys.org/resources?r=10>